



Tehnologii WEB

Curs 6

JavaScript

Cod Teams: **6gyjd1x**

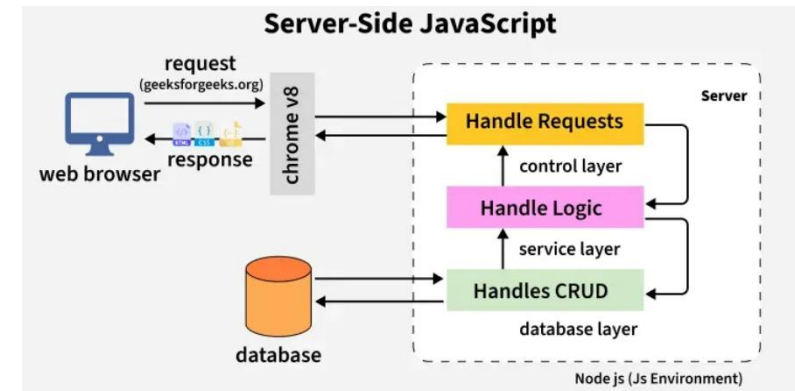
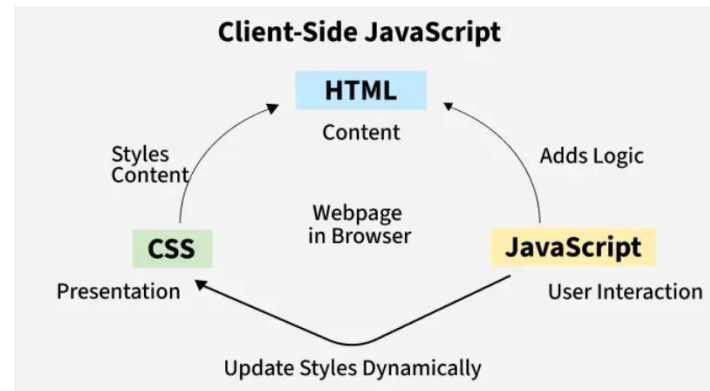
Anul III, semestrul II

2026

JavaScript

JavaScript este un limbaj de programare folosit pentru a crea conținut dinamic pentru site-uri web. Este un limbaj de programare ușor, multiplatformă și cu un singur fir de execuție. Este un limbaj interpretat care execută cod linie cu linie, oferind mai multă flexibilitate.

- **Partea de client:** Pe partea de client, JavaScript funcționează împreună cu HTML și CSS. HTML adaugă structură unei pagini web, CSS o stilizează, iar JavaScript o dă viață permițând utilizatorilor să interacționeze cu elementele de pe pagină, cum ar fi acțiuni la clicurile pe butoane, completarea formularelor și afișarea animațiilor. JavaScript pe partea de client este executat direct în browserul utilizatorului.
- **Partea de server :** Pe partea de server (pe serverele web), JavaScript este utilizat pentru a accesa bazele de date, gestionarea fișierelor și funcțiile de securitate pentru a trimite răspunsuri către browsere.

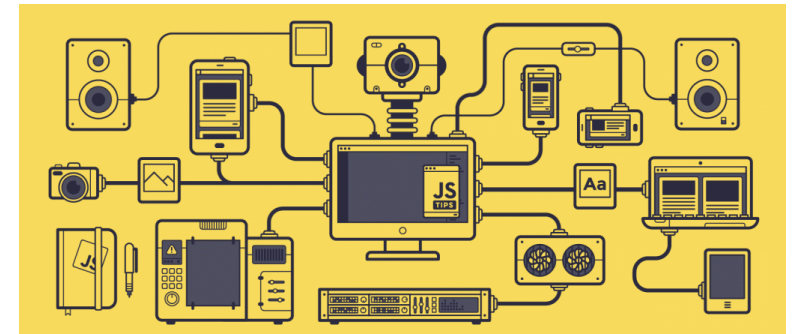


Unde se folosește JavaScript:



Principalele domenii de utilizare a JavaScript:

- **Dezvoltare Web Frontend** (Interfața Utilizatorului): Creează elemente interactive pe site-uri, cum ar fi meniuri drop-down, hărți interactive, animații și validarea datelor introduse de utilizatori, fără a reîncărca pagina.
- **Dezvoltare Web Backend** (Server-side): Cu ajutorul Node.js, JavaScript este utilizat pentru a construi servere, gestiona baze de date și crea API-uri.
- **Aplicații Mobile**: Framework-uri precum React Native sau NativeScript permit crearea de aplicații native pentru iOS și Android folosind JavaScript.
- **Internet of Things (IoT)**: Se folosește pentru programarea dispozitivelor conectate, inclusiv senzori și plăci, precum Raspberry Pi sau Arduino.
- **Dezvoltare de Jocuri și Aplicații Desktop**: Utilizat pentru crearea de jocuri de browser și aplicații desktop, inclusiv smartwatch-uri.



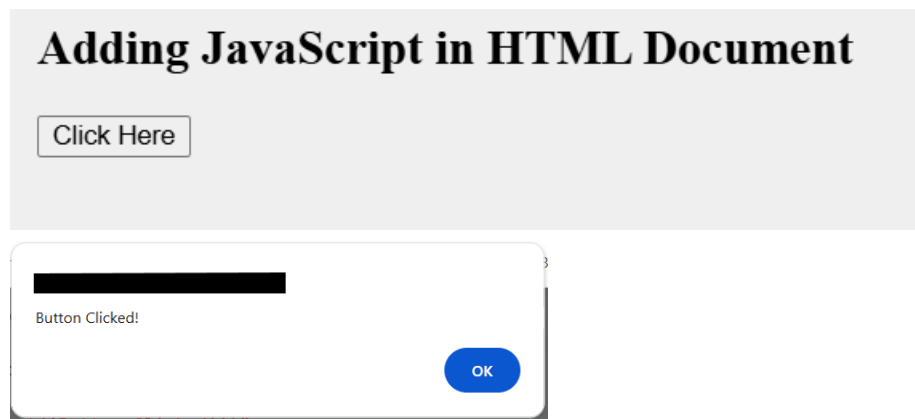
Adăugarea de JavaScript în documentul HTML

JavaScript poate fi inclus într-un document HTML pentru a adăuga interactivitate și comportament dinamic paginilor web. Acesta permite browserului să execute scripturi care pot modifica conținutul, gestiona evenimentele și comunica cu serverele.

- **JavaScript inline:** Cod scris direct în interiorul etichetelor HTML folosind attribute precum onclick.
- **JavaScript intern:** Script scris în interiorul etichetei `<script>` din fișierul HTML.
- **JavaScript extern:** JavaScript stocat într-un fișier .js separat și legat folosind eticheta `<script src="file.js"></script>`.

JavaScript în linie: scrierea codului JavaScript direct în interiorul elementului HTML folosind attributele **onclick**, **onmouseover** sau alte attribute de gestionare a evenimentelor.

```
<html>
<head></head>
<body>
  <h2>
    Adding JavaScript in HTML Document
  </h2>
  <button onclick="alert('Button Clicked!')">
    Click Here
  </button>
</body>
</html>
```



JavaScript intern (în cadrul etichetei <script>): scrierea codului JavaScript în interiorul etichetei <script> din fișierul HTML. Acesta este cunoscut sub numele de JavaScript intern și este plasat de obicei în secțiunea <head> sau <body> a documentului HTML.

1. Cod JavaScript în interiorul etichetei <head>

Plasarea codului JavaScript în secțiunea <head> a unui document HTML asigură încărcarea și executarea scriptului pe măsură ce pagina se încarcă. Acest lucru este util pentru scripturile care trebuie inițializate înainte de redarea conținutului paginii.

```
<html>
<head>
  <script>
    function myFun() {
      document.getElementById("demo")
        .innerHTML = "Ai Castigat!";  }
  </script></head><body>
  <h2>
    Adauga cod aici
  </h2>
  <h3 id="demo" style="color:green;">
    AICI APASA!
  </h3>
  <button type="button" onclick="myFun()">
    Click Here
  </button></body></html>
```

Adauga cod aici

AICI APASA!

Click Here

Adauga cod aici

Ai Castigat!

Click Here

2. Cod JavaScript în interiorul etichetei <body>: JavaScript poate fi plasat și în secțiunea <body> a unei pagini HTML. De obicei, scripturile plasate la sfârșitul secțiunii <body> se încarcă după conținut, ceea ce poate fi util dacă scriptul depinde de încărcarea completă a DOM-ului.

```
<html>
<head></head>
<body>
  <h4>
    Introdu Cod
  </h4>
  <h6 id="demo" style="color:blue;">
    Apasa aici!
  </h6>
  <button type="button" onclick="myFun()">
    Click Here
  </button>  <script>
    function myFun() {
      document.getElementById("demo")
        .innerHTML = "Ai castigat si acum!";
    }
  </script></body></html>
```

Introdu Cod

Apasa aici!

Click Here

Introdu Cod

Ai castigat si acum!

Click Here

JavaScript extern (folosind fișier extern): Pentru proiecte mai mari sau când reutilizați scripturi în mai multe fișiere HTML, puteți plasa codul JavaScript într-un fișier .js extern. Acest fișier este apoi legat de documentul HTML folosind atributul src dintr-o etichetă <script>.

```
<html>
<head>
  <script src="script.js"></script>
</head>
<body>
  <h2>
    Cod JavaScript extern
  </h2>
  <h3 id="demo" style="color:rgb(0, 128, 111);">
    Apasa aici!
  </h3>
  <button type="button" onclick="myFun()">
    Click Here
  </button>
</body>
</html>
```

```
JS script.js > myFun
1
2 function myFun () {
3   document.getElementById('demo')
4     .innerHTML = 'Ai ajuns la final!'
5 }
```

Cod JavaScript extern

Apasa aici!

Click Here

Cod JavaScript extern

Ai ajuns la final!

Click Here

Avantajele JavaScript-ului extern

- *Timpi de încărcare a paginilor mai rapizi:* Fișierele JavaScript externe memorate în cache nu trebuie reîncărcate de fiecare dată când pagina este vizitată, ceea ce poate accelera timpii de încărcare.
- *Lizibilitate și întreținere îmbunătățite:* Păstrarea separată a HTML-ului și JavaScript-ului face ca ambele să fie mai ușor de citit și de întreținut.
- *Separarea aspectelor:* Prin separarea HTML (structură) și JavaScript (comportament), codul devine mai curat și mai modular.
- *Reutilizarea codului:* Un fișier JavaScript extern poate fi legat la mai multe fișiere HTML, reducând redundanța și facilitând actualizările.



JavaScript asincron și amânat

JavaScript poate fi încărcat asincron sau amânat pentru a optimiza performanța paginii, în special pentru scripturile mai mari.

În mod implicit, JavaScript blochează redarea paginii HTML până când aceasta este complet încărcată, dar utilizarea funcției asincron sau a amânării poate ajuta la îmbunătățirea timpilor de încărcare.

1. *Atribut asincron:* Atributul **async** încarcă scriptul asincron, ceea ce înseamnă că scriptul va fi descărcat și executat imediat ce este disponibil, fără a bloca pagina.

```
<script src="script.js" async></script>
```

2. *Atribut de amânare:* Atributul ``defer`` întârzie execuția scriptului până când întregul document HTML a fost analizat. Acest lucru este util în special pentru scripturile care manipulează DOM-ul.

```
<script src="script.js" defer></script>
```

Cum se face referire la fișiere JavaScript externe?

Prin utilizarea unei adrese URL complete:

```
sursă = "https://www.geeksforgeek.org/js/script.js"
```

Prin utilizarea unei căi de fișier:

```
src = "/js/script.js"
```

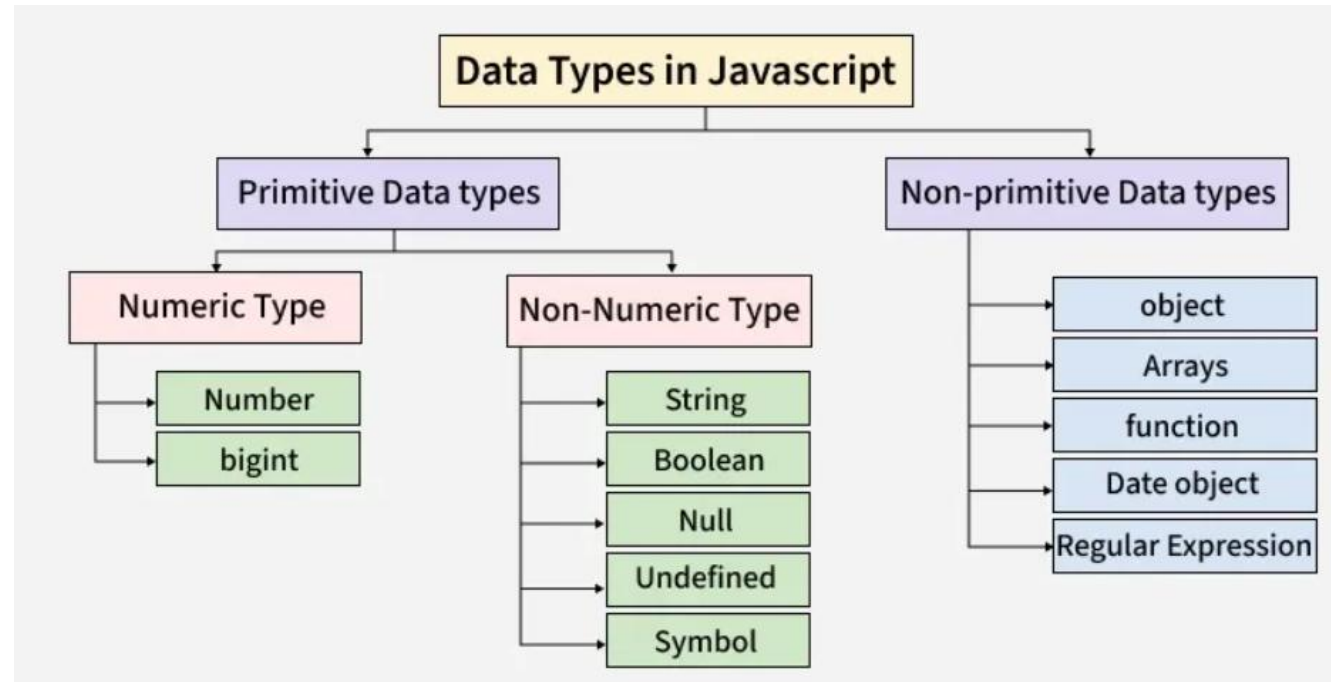
Fără a utiliza nicio cale:

```
src = "script.js"
```

Variabile și tipuri de date în JavaScript

Variabilele sunt folosite pentru a stoca valori de date care pot fi utilizate și modificate într-un program. JavaScript acceptă diferite tipuri de date pentru a gestiona numere, text și alte tipuri de informații.

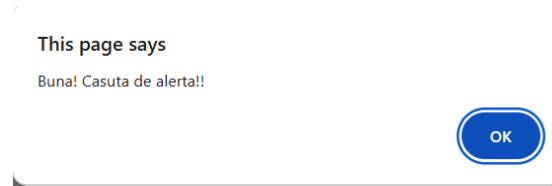
- **Variabile:** Declarate folosind var, let și const pentru a stoca valorile datelor.
- **Tipuri de date primitive:** Include număr, șir de caractere, boolean, nul, nedefinit, BigInt și simbol.
- **Tipuri de date non-primitive:** Include obiecte, matrice și funcții utilizate pentru stocarea datelor complexe.



Variabile: O variabilă este ca un container care conține date ce pot fi reutilizate sau actualizate ulterior în program. În JavaScript, variabilele sunt declarate folosind cuvintele cheie **var**, **let** sau **const**.

a) **var**: este folosit pentru a declara o variabilă. Aceasta are un comportament de tip funcție-scoped sau global-scoped.

```
function myFun () {  
  var mesaj = "Buna! Casuta de alerta!!";  
  document.getElementById('demo')  
    .innerHTML = n+ ' ', 'Ai ajuns la final!'  
  alert(mesaj);  
  
  var n = 5;  
  console.log(n);  
  var n = 20; // reatribuirea este permisa  
  console.log(n);  
}
```



5

20

b) **let**: declară variabile cu scop de bloc, permițând actualizări, dar nu și redeclararea în cadrul aceluiași bloc.

```
let n = 10;  
n = 20; // Variabila poate fi actualizata  
console.log(n)
```

20

c) **const**: declară variabile cu scop de bloc care nu pot fi reatribuite după atribuirea lor inițială.

```
const n = 100;  
//n = 200; Acesta v-a genera o eroare  
console.log(n)
```

100

```
const n = 100;  
n = 200; //Acesta v-a genera o eroare  
console.log(n)
```

✖ Uncaught TypeError: Assignment to constant variable.

Diferența dintre cuvintele cheie var, let și const în JavaScript

```
// var exemplu
var x = 10;
var x = 20;    // Redeclarare permisa
x = 30;        // Actualizare permisa
console.log(x); // Afisare: 30

// let exemplu
let y = 10;
// let y = 20; // Redeclarare NU este permisa
y = 25;        // Actualizare permisa
console.log(y); // Afisare: 25

// const exemplu
const z = 10;
// z = 20;     // Redeclarare NU este permisa
console.log(z); // Afisare: 10

// Demonstratie a domeniului de aplicare a blocului
if (true) {
  var a = 1;
  let b = 2;
  const c = 3;
}

console.log(a); // Afiseaza (var este o functie cu scop global)
// console.log(b); // Eroare (let este in domeniul de aplicare al unui bloc)
// console.log(c); // Eroare (const este in domeniul de aplicare al unui bloc)
```

30
25
10
1

Tipuri de date

JavaScript are tipuri de **date primitive** (string, number, boolean, null, undefined, symbol, bigint) și tipuri de referință (obiecte, array-uri, funcții). Acestea permit stocarea și manipularea diferitelor forme de informații, cum ar fi text, cifre, valori logice sau structuri complexe de date, fiind esențiale pentru dezvoltarea web.

Acestea sunt imutabile (nu pot fi modificate direct) și stochează o singură valoare.

- **Number:** Reprezintă atât numere întregi, cât și numere cu virgulă. Exemplu: *let varsta = 25;* sau *let pret = 99.99;*
- **String:** Șiruri de caractere folosite pentru text, delimitate de ghilimele.

Exemplu: *let nume = "Alex";* sau *let salut = 'Bună ziua';*

- **Boolean:** Poate avea doar două valori: true (adevărat) sau false (fals).

Exemplu: *let esteMajor = true;*

- **Undefined:** O variabilă declarată care nu a primit încă o valoare are automat acest tip.

Exemplu: *let x; // typeof x este "undefined"*

- **Null:** Reprezintă absența intenționată a oricărei valori. Exemplu: *let dateUtilizator = null;*
- **Symbol:** Folosit pentru a crea identificatori unici pentru proprietățile obiectelor.

Exemplu: *const id = Symbol('descriere');*

- **BigInt:** Folosit pentru numere întregi foarte mari care depășesc limita tipului Number.

Exemplu: *let numarGigant = 9007199254740991n;*

```
let a = 10; // decimal - implicit
let b = 0x10; // hexadecimal
let c = 0o10; // octal
let d = 0b10; // binary
```

```
console.log(a); // -> 10
console.log(b); // -> 16
console.log(c); // -> 8
console.log(d); // -> 2
```

```
let x = 5e3;
let y = 723e-4;
console.log(x); // 5000
console.log(y); // 0.0723
```

```
let a = 1 / 0;
let b = -Infinity;
```

```
console.log(a); // -> Infinity
console.log(b); // -> -Infinity
console.log(typeof a); // -> number
console.log(typeof b); // -> number
```

```
let s = "string";
let n = s * 5;
console.log(n); // -> NaN
console.log(typeof n); // -> number
```

Tipuri de date

Tipuri de Referință (Obiecte): Acestea pot stoca colecții de date sau entități complexe.

- **Object:** O colecție de perechi cheie-valoare. Exemplu: *let masina = { marca: "Dacia", model: "Logan" };*
- **Array** (Tablou): O listă ordonată de valori. Exemplu: *let culori = ["rosu", "verde", "albastru"];*
- **Function:** Funcțiile sunt, de asemenea, considerate obiecte în JavaScript.

Exemplu: *function salut() { return "Salut!"; }*

Pentru verificarea tipului unei variabile se folosește operatorul **typeof**.

Operatorul **typeof** este un operator unar care returnează un sir de caractere care reprezintă tipul de date al argumentului dat. Argumentul poate fi un literal sau o variabilă.

Valorile posibile care pot fi returnate: undefined, object, Boolean, number, bigint, string, symbol, function.

```
console.log(typeof "A"); // string
console.log(typeof 4); // number
console.log(typeof false); // boolean
console.log(typeof x); // undefined
console.log(typeof {}); // object
let a = 7.5;
console.log(typeof a); //number
```

Operatori JavaScript

Operatorii JavaScript sunt simboluri sau cuvinte cheie folosite pentru a efectua operații asupra valorilor și variabilelor. Aceștia sunt elementele constitutive ale expresiilor JavaScript și pot manipula datele în diverse moduri.

1. Operatori aritmetici JavaScript: efectuează calcule matematice precum adunarea, scăderea, înmulțirea etc.

- + adună două numere.
- - scade al doilea număr din primul.
- * înmulțește două numere.
- / împarte primul număr la al doilea.

```
const sum = 5 + 3; // Adunarea
const diff = 10 - 2; // Scaderea
const p = 4 * 2; // Inmultirea
const q = 8 / 2; // Impartirea
console.log(sum, diff, p, q);
```

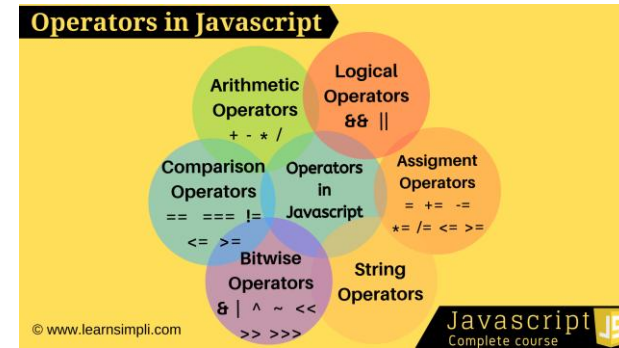
8
8
8
4

2. Operatori de atribuire JavaScript: sunt utilizați pentru a atribui valori variabilelor. De asemenea, pot efectua operații precum adunarea sau înmulțirea în timp ce atribuie valoarea.

- = atribuie o valoare unei variabile.
- += adună și atribuie rezultatul variabilei.
- *= înmulțește și atribuie rezultatul variabilei.

```
let n = 10;
n += 5;
n *= 2;
console.log(n);
```

30



3. Operatori de comparație JavaScript: compară două valori și returnează o valoare booleană (adevărat sau fals). Aceștia sunt utili pentru luarea deciziilor în instrucțiunile condiționale.

- > verifică dacă valoarea din stânga este mai mare decât cea din dreapta.
- === verifică egalitatea strictă (atât tipul, cât și valoarea).
- Alți operatori includ <, <=, >= și !==.

```
console.log(10 > 5);  
console.log(10 === "10");
```

adevărat

fals

4. Operatori logici JavaScript: sunt utilizați în principal pentru a efectua operații logice care determină egalitatea sau diferența dintre valori.

- && returnează valoarea true dacă ambii operanzi sunt true.
- || returnează valoarea „true” dacă cel puțin un operand este „true”.
- ! neagă valoarea booleană.

```
const a = true, b = false;  
console.log(a && b); // Logic AND  
console.log(a || b); // Logic OR
```

fals

adevărat

5. Operatori JavaScript pe biți: efectuează operații asupra reprezentărilor binare ale numerelor.

- & efectuează o operație AND la nivel de bit.
- | efectuează o operație SAU la nivel de bit.
- ^ efectuează o operație XOR la nivel de bit.
- ~ execută o operație NU la nivel de bit.

```
const res = 5 & 1; // AND pe bit  
console.log(res);
```

1

6. Operatorul ternar (Ternary) din JavaScript: este un operator condițional care evaluează o condiție și returnează una dintre cele două valori, în funcție de dacă condiția este adevărată sau falsă. Simplifică luarea deciziilor în cod, făcându-l mai concis și mai ușor de citit.

condiție ? expresie1 : expresia2 evaluează expresia1 dacă condiția este adevărată, altfel evaluează expresia2.

```
const age = 18;  
const status = age >= 18 ? "Adult" : "Minor";  
console.log(status);
```

Adult

7. Operatorul virgulă (,): evaluează în principal operanzii săi secvențial de la stânga la dreapta și returnează valoarea operandului din dreapta.

- Fiecare expresie este evaluată de la stânga la dreapta.
- Rezultatul final al expresiei este valoarea din dreapta.

```
let n1, n2  
const res = (n1 = 1, n2 = 2, n1 + n2);  
console.log(res);
```

3

8. Operatori unari JavaScript: operează asupra unui singur operand (de exemplu, incrementare, decrementare).

- ++ incrementează valoarea cu 1.
- -- scade valoarea cu 1.
- typeof returnează tipul unei variabile.

```
let x = 5;  
console.log(++x); // Pre-increment  
console.log(x--); // Post-decrement (Output: 6, then x becomes 5)
```

9. Operatori relaționali JavaScript: sunt utilizați pentru a compara operanzii și a determina relația dintre aceștia. Aceștia returnează o valoare booleană (adevărat sau fals) pe baza rezultatului comparației.

- **in** verifică dacă o proprietate există într-un obiect.
- **instanceof** verifică dacă un obiect este o instanță a unui constructor.

```
const obj = { length: 10 };  
console.log("length" in obj);  
console.log([] instanceof Array);
```

true

true

10. Operatori BigInt în JavaScript: permit calcule cu numere dincolo de intervalul întreg sigur.

- Operații precum adunarea, scăderea și înmulțirea funcționează cu BigInt.
- Se folosește sufixul **n** pentru a desemna literalii BigInt.

```
const big1 = 123456789012345678901234567890n;  
const big2 = 987654321098765432109876543210n;  
console.log(big1 + big2);
```

1111111110111111111011111111100

11. Operatori de șiruri JavaScript: includ concatenarea (+) și atribuirea de concatenare (+=), utilizați pentru a uni șiruri de caractere sau a combina șiruri de caractere cu alte tipuri de date.

- + concatenează șiruri de caractere.
- += se adaugă la un șir de caractere existent.

```
const s = "Hello" + " " + "World";  
console.log(s);
```

Hello World

Declarații de flux de control JavaScript

Instrucțiunile de control al fluxului de cod din JavaScript controlează ordinea în care este executat codul. Aceste instrucțiuni vă permit să luați decizii, să repetați sarcini și să treceți de la o parte a unui program pe baza unor condiții specifice.

1. Instrucțiuni Condiționale (Decizionale): Acestea permit executarea unor blocuri de cod doar dacă o anumită condiție este îndeplinită

- **if...else:** Execută un bloc de cod dacă condiția este adevărată; opțional, un alt bloc (else) dacă este falsă.
- **else if:** Permite testarea mai multor condiții succesive.
- **switch:** O alternativă mai lizibilă la multiple if-uri, utilă când se compară aceeași variabilă cu mai multe valori posibile.

```
let nota = 85;

if (nota >= 90) {
  console.log("Calificativ: A");
} else if (nota >= 80) {
  console.log("Calificativ: B");
} else {
  console.log("Calificativ: C");
}
```

Calificativ: B

```
let fruct = "mar";

switch (fruct) {
  case "banana":
    console.log("Este o banană.");
    break;
  case "mar":
    console.log("Este un măr.");
    break;
  default:
    console.log("Fruct necunoscut.");
}
```

Este un măr.

2. Instrucțiuni Repetitive (Loops): Sunt folosite pentru a rula același cod de mai multe ori până când o condiție nu mai este validă.

- **for:** Ideal atunci când știi exact de câte ori vrei să repeți acțiunea.
- **while:** Repetă codul atât timp cât condiția specificată rămâne adevărată.
- **do...while:** Similar cu while, dar garantează că blocul de cod este executat cel puțin o dată, deoarece verificarea condiției are loc la final.
- **for...in / for...of:** Variante specializate pentru a parcurge proprietățile unui obiect sau elementele unei colecții (precum un Array).

```
for (let i = 1; i <= 3; i++) {  
  console.log("Iterația numărul: " + i);  
}
```

```
Iterația numărul: 1  
Iterația numărul: 2  
Iterația numărul: 3
```

```
let count = 1;  
while (count < 3) {  
  console.log("Număr: " + count);  
  count++;  
}
```

```
Număr: 1  
Număr: 2
```

```
let i = 5;  
do {  
  console.log("Se execută o dată chiar dacă 5 nu e < 5");  
} while (i < 5);
```

```
Se execută o dată chiar dacă 5 nu e < 5
```

3. **Gestionarea Excepțiilor:** permit controlul programului în cazul apariției unor erori neprevăzute.

- **try...catch...finally:** Codul din **try** este monitorizat; dacă apare o eroare, execuția trece imediat la **catch**. Blocul **finally** se execută indiferent de rezultat.
- **throw:** Permite crearea de erori personalizate de către programator.

```
try {  
  console.log(variabilaInexistenta);  
} catch (eroare) {  
  console.log("A apărut o eroare: " + eroare.message);  
}
```

A apărut o eroare: variabilaInexistenta is not defined

4. Controlul Salturilor

- **break:** Oprește imediat execuția unei bucle sau a unui switch.
- **continue:** Sarem peste iterația curentă a unei bucle și trece direct la următoarea.
- **return:** Oprește execuția unei funcții și returnează o valoare

```
for (let i = 1; i <= 5; i++) {  
  if (i === 2) continue; // Sare peste 2  
  if (i === 4) break;    // Oprește bucla la 4  
  console.log(i);  
}
```

1

3

Domeniul de aplicare al variabilelor în JavaScript

Domeniul de aplicare determină unde poate fi accesată sau utilizată o variabilă într-un program JavaScript. Acesta ajută la controlul vizibilității și duratei de viață a variabilelor în diferite părți ale codului.

1. Domeniu de aplicare global și local

O **variabilă globală** se referă la o variabilă declarată în afara oricărei funcții sau bloc, deci poate fi utilizată oriunde în program, atât în interiorul funcțiilor, cât și în codul principal.

```
// Variabilă globală accesată din cadrul unei funcții
```

```
const x = 10;
```

```
function fun1() {  
    console.log(x);  
}
```

```
fun1();
```

10

Explicație: În program, variabilele sunt în afara funcției și acum putem accesa aceste variabile de oriunde în programul JavaScript.

```
// Declararea unei variabile globale
```

```
fie x = 10 ;
```

```
funcție func () {
```

```
// Declararea unei variabile locale
```

```
fie y = 20 ;
```

```
// Accesarea la nivel local și global
```

```
// variabile
```

```
console.log ( x , " , " , y ) ;
```

```
}
```

```
funcție ();
```

O **variabilă locală** este o variabilă declarată în interiorul unei funcții, fiind accesibilă doar în cadrul acelei funcții. Nu poate fi utilizată în afara funcției.

Funcțiile și obiectele sunt, de asemenea, variabile în JavaScript.

```
function fun2(){  
    // Această variabilă este locală pentru fun2() și  
    // nu poate fi accesat în afara acestei funcții  
    let x = 10;  
    console.log(x);  
}  
  
fun2();
```

Aici, codul definește o funcție *fun2* cu o variabilă locală *x*, accesibilă doar în interiorul funcției, și afișează valoarea acesteia atunci când funcția este apelată.

Variabilele declarate cu ``let`` și ``const`` au fie domeniul de aplicare al unui bloc, fie domeniul de aplicare global.

2. Bloc și domeniu lexical

În JavaScript, **domeniul de aplicare** al unui bloc se referă la variabilele declarate cu `let` sau `const` în interiorul unui `{ }` bloc. Aceste variabile sunt accesibile numai în interiorul blocului respectiv și nu în afara acestuia.

Variabilele declarate cu `var` nu au domeniul de aplicare al unui bloc. O variabilă `var` declarată în interiorul unei funcții este accesibilă în întreaga funcție, indiferent de orice blocuri (cum ar fi instrucțiuni `if` sau bucle `for`) din cadrul funcției. Dacă `var` este declarat `used` în afara oricărei funcții, aceasta creează o variabilă globală.

```
{
```

```
// Variabila poate fi accesibilă în interiorul și în afara domeniului de aplicare al blocului
```

```
var x = 10;
```

```
// let, const Accesibil doar în domeniul de aplicare al blocului
```

```
const y = 20;
```

```
let z = 30;
```

```
console.log(x);
```

```
console.log(y);
```

```
console.log(z);
```

```
}
```

```
console.log(x);
```

10

20

În **domeniu lexical**: Variabila este declarată în interiorul funcției și poate fi accesată doar în interiorul blocului sau blocului imbricat respectiv și se numește domeniu de aplicare lexical.

```
function func1() {  
  const x = 10;  
  
  function func2() {  
    const y = 20;  
    console.log(`${x} ${y}`);  
  }  
  
  func2();  
}  
  
func1();
```

Acest cod demonstrează domeniul de aplicare lexical, unde func2 accesează variabila x din func1 și afișează „10 20”.

10 20

Domeniul de aplicare al modulului se referă la variabilele și funcțiile accesibile doar în cadrul unui anumit modul JavaScript. Acesta ajută la menținerea codului organizat și previne ca variabilele să afecteze domeniul de aplicare global.

- math.js are variabile și funcții în domeniul de aplicare al modulului său.
- Acestea sunt accesibile în alte fișiere doar atunci când folosim exportul și importul.

```
// math.js (module file)  
export const number = 10;  
  
export function add(a, b) {  
  return a + b;  
}
```

```
// main.js (another file)  
import { number, add } from "./math.js";  
  
console.log(number);    // 10  
console.log(add(5, 3)); // 8
```

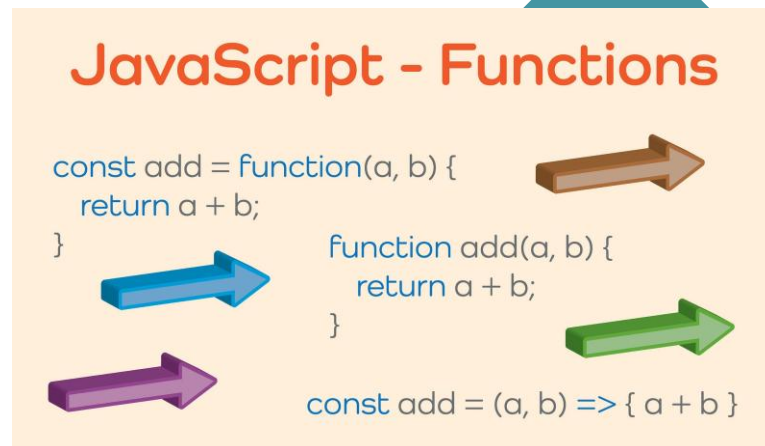
Funcții în JavaScript

Funcțiile din JavaScript sunt blocuri de cod reutilizabile, concepute pentru a îndeplini sarcini specifice. Acestea vă permit să organizați, să reutilizați și să modularizați codul. Acesta poate primi intrări, poate efectua acțiuni și poate returna ieșiri.

```
function greet(name) { // 'name' este un parametru
  console.log("Hello " + name);
}

greet("Alice"); // "Alice" este argumentul
```

- Parametru: nume (substituit în interiorul funcției).
- Argument: „Alice” (valoare reală dată la momentul apelului).



Parametrii impliciți

- Parametrii impliciți sunt utilizați atunci când nu este furnizat niciun argument în timpul apelului funcției.
- Dacă nu se transmite nicio valoare, funcția folosește automat valoarea implicită.

```
function greet(name = "Guest") {
  console.log("Hello, " + name);
}

greet();
greet("AAAA");
```

Returnează: Hello, Guest

Hello, AAAA

Instrucțiunea **Return** utilizată pentru a trimite înapoi un rezultat de la o funcție. Când return se execută, funcția se oprește în acel moment. Valoarea returnată poate fi stocată într-o variabilă sau utilizată direct.

```
function add(a, b) {  
  return a + b; // returns the sum  
}
```

```
let result = add(5, 10);  
console.log(result);
```

15

Tipuri de funcții principale în JavaScript

1. **Funcția identificată prin nume:** O funcție care are propriul nume atunci când este declarată. Este ușor de reutilizat și de depanat deoarece numele apare în mesajele de eroare.

```
function greet() {  
  return "#####Hello!#####";  
}  
console.log(greet());
```

#####Hello!#####

2. **Funcția anonimă:** O funcție care nu are un nume. De obicei, este atribuită unei variabile sau este utilizată ca o funcție de apel invers. Deoarece nu are nume, nu poate fi apelată direct.

```
const greet = function() {  
  return "Salutare!";  
};  
console.log(greet());
```

Salutare!

Când atribuieți o funcție (poate fi denumită sau anonimă) unei variabile. Funcția poate fi apoi utilizată prin apelarea acelei variabile.

```
const add = function(a, b) {  
  return a + b;  
};  
console.log(add(2, 3));
```

5

```
const square = n => n * n;  
console.log(square(4));
```

16

3. Expresie de funcție invocată imediat (IIFE): Expresiile de funcție invocate imediat (IIFE) sunt funcții JavaScript care se execută imediat după ce sunt definite. De obicei, acestea sunt folosite pentru a crea un domeniu de aplicare local pentru variabile, pentru a le împiedica să polueze domeniul de aplicare global.

Sintaxă:

Exemple:

```
(function () {  
  // Logica funcției aici.  
})();
```

```
(function() {  
  // IIFE code block  
  var localVar = 'This is a local variable';  
  console.log(localVar); // Output: This is a local variable  
})();
```

```
var result = (function() {  
  var x = 10;  
  var y = 20;  
  return x + y;  
})();  
  
console.log(result); // Output: 30
```

4. Funcția callback: este o funcție care este transmisă ca argument unei alte funcții și executată ulterior. O funcție poate accepta o altă funcție ca parametru.

```
function greet(name, callback) {  
  console.log("Hello, " + name);  
  callback();  
}
```

```
function sayBye() {  
  console.log("Goodbye!");  
}
```

```
greet("Ajay", sayBye);
```

Hello, Ajay

Goodbye!

```
function num(n, callback) {  
  return callback(n);  
}
```

```
const double = (n) => n * 2;
```

```
console.log(num(5, double));
```

10

5. Funcție cu constructor: Un tip special de funcție folosită pentru a crea mai multe obiecte cu aceeași structură. Se apelează cu “new” .

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}  
  
const user = new Person("Neha", 22);  
console.log(user.name);
```

Neha

6. Funcție recursivă: O funcție care se apelează singură până când este îndeplinită o condiție. Foarte utilă pentru probleme precum factoriale, Fibonacci sau parcurgeri de arbori.

```
function factorial(n) {  
  if (n === 0) return 1;  
  return n * factorial(n - 1);  
}  
console.log(factorial(5));
```

120

7. Funcție de ordin superior: O funcție care fie primește o altă funcție ca parametru, fie returnează o altă funcție. Acestea sunt comune în JavaScript (de exemplu, map, filter, reduce).

```
function multiplyBy(factor) {  
  return function(num) {  
    return num * factor;  
  };  
}
```

```
const double = multiplyBy(2);  
console.log(double(5));
```

10

Studiu individual

1. Sa se realizeze o aplicatie “Contor de click-uri” (Crește un număr de fiecare dată când apeși pe buton). Foloseste HTML, CSS si JavaScript.

2. Sa se realizeze o aplicatie “Schimba culoarea paginii” (Apăsând pe buton se schimbă culoarea paginii (alb-albastru-rosu-galben-portocaliu-verde-mov.). Foloseste HTML, CSS si JavaScript.
3. Sa se realizeze o aplicatie “Mini-calculator” (Operații de +, -, *, / de n numere). Foloseste HTML, CSS si JavaScript.